

Exception Handling in Java



HEALTHCARE DOMAIN

INTERMEDIATE LEVEL

A comprehensive guide to Throwable, Error, Checked, Unchecked, Custom Exceptions, and Exception Chaining — applied to Patient Management, Medical Devices, EMR/EHR, and Insurance Claims systems.

*Developed by **Talenciaglobal***

Why Exception Handling Is Life-Critical in Healthcare

- ❌ Healthcare systems cannot crash mid-surgery. Patient data integrity is life-critical. Regulatory compliance (HIPAA, GDPR) is mandatory.

98.6%

Uptime Required

EMR/EHR systems in ICUs must maintain near-perfect uptime — a single unhandled exception can interrupt life-support monitoring.

\$10M+

Average HIPAA Fine

Data breaches caused by improper exception handling exposing PHI can result in multi-million dollar regulatory penalties.

70%

Bugs from Null Refs

Studies show approximately 70% of production Java crashes stem from `NullPointerException` — the most preventable unchecked exception.

Exception handling is not merely a coding convention — in healthcare software, it is the difference between a recoverable system event and a catastrophic patient safety incident.



What Is Exception Handling? — The Hospital Emergency Protocol

Normal Hospital Flow

Patient checks in → Doctor examines → Treatment given → Patient discharged.

This is the **try block** — the expected, happy-path execution of your program.

Emergency (Exception) Flow

Patient's heart stops → **Code Blue team rushes in** (catch block) → Patient stabilized → Back to normal flow.

Without exception handling, the entire hospital shuts down when one patient has an emergency. With it, problems are isolated, handled, and the system continues.

Healthcare Analogy Mapping

try block

Normal medical procedure (surgery, medication administration)

catch block

Emergency response team (crash cart, rapid response unit)

finally block

Always-cleanup: sterilize equipment, update medical records

throw

Doctor recognizing and formally reporting an emergency

throws

Hospital declaring "We can handle cardiac emergencies"

Formal Definition

Exception handling is a mechanism in Java that separates error-handling code from normal business logic, allowing programs to detect, report, and recover from runtime anomalies without crashing. The **Throwable** class is the superclass of all errors and exceptions in Java.



Separation of Concerns

Business logic remains clean and readable. Error-handling code is isolated in dedicated catch blocks, improving maintainability of EMR systems.



Audit Trail

Structured exception handling enables comprehensive logging for HIPAA compliance — every error event is captured with context and timestamp.



Graceful Recovery

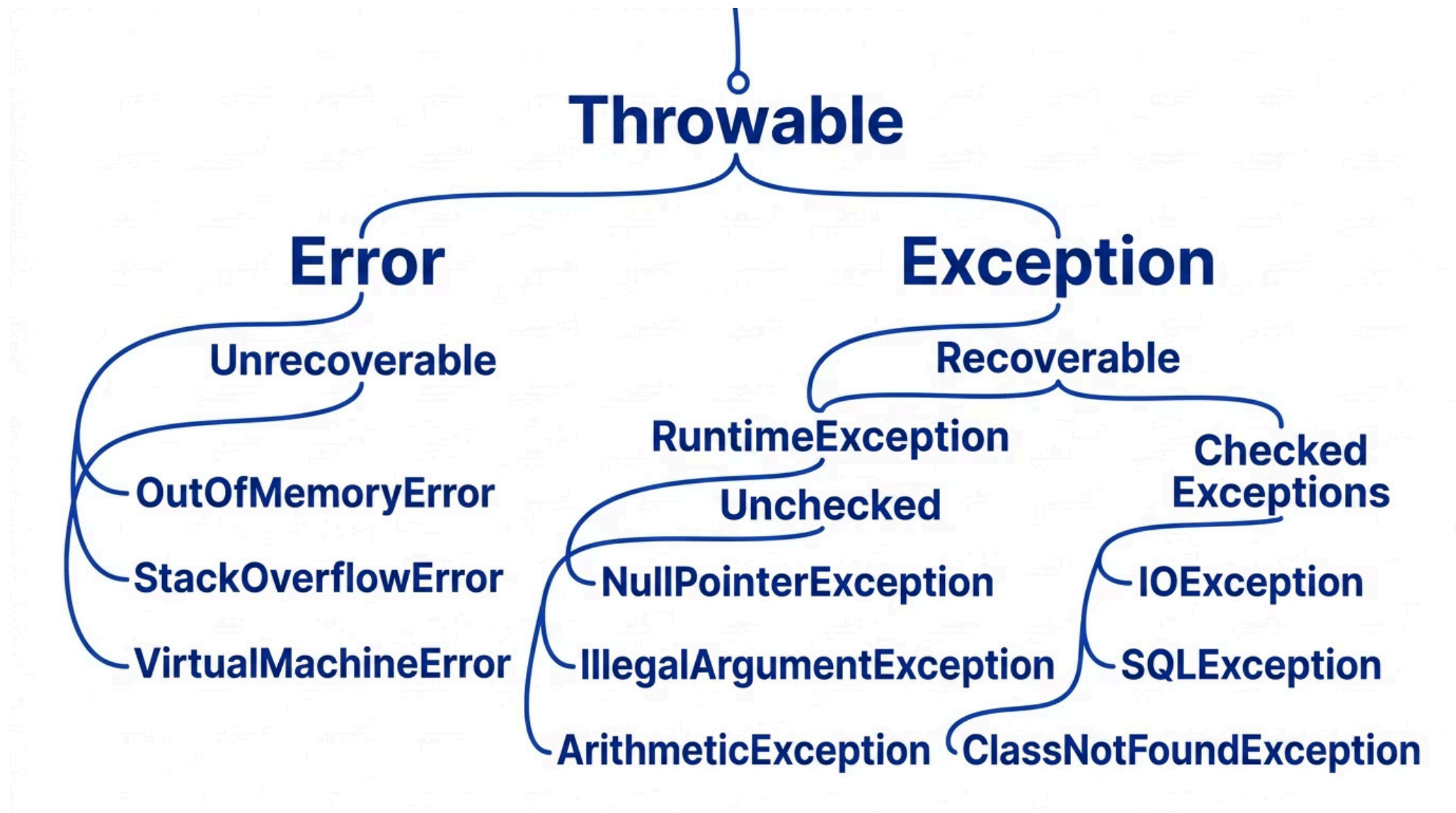
Systems can recover from predictable failures — a missing patient record triggers a user-friendly prompt rather than a system crash.



Layered Architecture

Exception chaining preserves root cause across service layers — from database connection failures up to the clinical decision API.

The Throwable Hierarchy



The Throwable hierarchy defines the complete taxonomy of Java runtime anomalies. Every exception or error in Java ultimately extends **Throwable**, which implements **Serializable** and carries a detail message, cause, and stack trace. Understanding this hierarchy is the foundation of writing robust healthcare software.

Throwable Classification — Detailed Reference

Type	Superclass	Compiler Checks?	Examples	Healthcare Example
Error	Error	No	OutOfMemoryError, StackOverflowError, NoClassDefFoundError	Hospital power outage, MRI machine firmware crash
Checked Exception	Exception (not RuntimeException)	Yes — must handle or declare	IOException, SQLException, ClassNotFoundException	Missing patient record in database, network failure to insurance system
Unchecked Exception	RuntimeException	No	NullPointerException, IllegalArgumentException, ArithmeticException	Null patient ID passed to admission system, invalid dosage calculation

 Industry statistic: According to a 2022 analysis of healthcare software incidents, **43% of critical system failures** in hospital information systems were attributable to unhandled or improperly handled exceptions — underscoring the importance of this classification.

Checked Exceptions — Compile-Time Safety

Definition

Checked exceptions represent **predictable, recoverable** conditions that a well-written application should anticipate and handle. The compiler **forces** you to handle them via try-catch or declare them with throws.

Healthcare Use Case

Patient record not found in database — an expected scenario that can be recovered by prompting the user to re-enter the patient ID. This is not a bug; it is a foreseeable business condition.

- ✓ Checked exceptions enforce defensive programming at compile time — the compiler acts as your first line of patient safety.

Three Mandatory Handling Options

1

Handle Locally

Wrap in try-catch and resolve the condition within the current method.

2

Declare with throws

Propagate the exception to the calling method using the throws keyword in the method signature.

3

Wrap as Unchecked

Catch and re-throw as RuntimeException when the checked exception cannot be meaningfully handled at the current layer.

Checked Exception — Code Example: Patient Record Retrieval

```
// Custom Checked Exception
class PatientNotFoundException extends Exception {
    private String patientId;
    public PatientNotFoundException(String patientId) {
        super("Patient not found with ID: " + patientId);
        this.patientId = patientId;
    }
    public String getPatientId() { return patientId; }
}

// Service declares checked exceptions
class HealthcareDataService {
    public Patient getPatientById(String patientId)
        throws SQLException, PatientNotFoundException {
        if (patientId == null || patientId.isEmpty()) {
            throw new IllegalArgumentException("Patient ID cannot be empty"); // Unchecked
        }
        Connection conn = DriverManager.getConnection("jdbc:hospital:db");
        if (patientId.equals("99999")) {
            throw new PatientNotFoundException(patientId);
        }
        return new Patient(patientId, "John Doe", 45);
    }
}

// Caller MUST handle checked exceptions
public class HospitalManagementSystem {
    public static void main(String[] args) {
        HealthcareDataService service = new HealthcareDataService();
        try {
            Patient patient = service.getPatientById("99999");
            System.out.println("Patient found: " + patient.getName());
        } catch (SQLException e) {
            System.err.println("Database error: " + e.getMessage());
            showUserError("Unable to connect to patient database. Please try again.");
        } catch (PatientNotFoundException e) {
            System.err.println("Patient lookup failed: " + e.getMessage());
            showUserError("Patient ID " + e.getPatientId() + " not found. Please verify.");
        }
    }
}
```

- ❏ Notice that **PatientNotFoundException** carries the patientId as a field — this enables precise, HIPAA-safe error messages that reference the ID without exposing sensitive PHI.

Unchecked Exceptions — Runtime Programming Errors

Definition

Unchecked exceptions extend **RuntimeException**. They represent **programming errors (bugs)** that should not occur in correct code. The compiler does NOT force handling — the developer is expected to prevent them through proper validation.

Healthcare Use Case

Passing a null patient object to a method that expects a valid patient — this is a **bug** that must be fixed at the source, not masked with a catch block.

 Best practice: Fix the bug. Do not catch `NullPointerException` — catching it hides the defect and creates unpredictable behavior in clinical workflows.

Common Unchecked Exceptions in Healthcare

NullPointerException

Accessing
`patient.getName()` when
patient reference is null

IllegalArgumentException

Prescribing a negative
dosage value to a
medication service

IllegalStateException

Attempting to start a
surgery that is already
in progress

ArithmeticException

Dividing total dosage by
patient weight of zero

ArrayIndexOutOfBoundsException

Accessing a bed index
beyond the ward
capacity array

ClassCastException

Casting a generic
Object to Patient when
it is actually a Device

Unchecked Exceptions — Prevention vs. Catching

❌ BAD: Catching NullPointerException

```
try {
    patient.getName();
} catch (NullPointerException e) {
    // This HIDES the bug!
    System.out.println("Patient is null");
}
```

Swallowing the NPE masks the defect. The root cause — a null patient reference — remains unfixed and will resurface unpredictably in production.

✅ GOOD: Prevent the Condition

```
// Guard clause — fail fast
public void processPatient(Patient patient) {
    if (patient == null) {
        throw new IllegalArgumentException(
            "Patient cannot be null");
    }
    // Safe to proceed
    String name = patient.getName();
}
```

Validate inputs at method entry. Throw a meaningful `IllegalArgumentException` that identifies the caller's contract violation immediately.

- 📌 Industry insight: The **fail-fast principle** is especially critical in healthcare software. A `NullPointerException` silently swallowed in a medication dispensing module could result in a patient receiving no medication — with no alert raised to clinical staff.

Errors — Unrecoverable JVM Failures

Definition

Errors represent **serious JVM-level problems** that a reasonable application should NOT attempt to catch. They indicate infrastructure-level failures beyond the application's control.

Healthcare Analogy

A hospital generator failure (power outage) — you do not write code to "catch" this. You invest in backup generators (external infrastructure redundancy). The application cannot self-heal from a JVM crash.

- ❌ Do NOT catch Errors in production healthcare systems. At most, log the event and initiate a graceful shutdown sequence.

Types of Errors in Healthcare Context

OutOfMemoryError

Loading an entire hospital's patient imaging archive into memory — JVM cannot allocate further objects. Requires infrastructure scaling, not code fixes.

StackOverflowError

Infinite recursion in a diagnostic algorithm — recursive call stack exceeds JVM limits. Fix the algorithm, not the error handler.

NoClassDefFoundError

Medical device driver class missing from classpath at runtime — deployment configuration error, not a recoverable condition.

VirtualMachineError

JVM internal failure — analogous to the hospital's core infrastructure collapsing. Requires system restart and incident investigation.

try-catch-finally — Complete Healthcare Example

The following example demonstrates a complete **Medical Imaging Processing** workflow using all three blocks of the exception handling construct:

```
public class MedicalImageProcessor {
    public static void processMRI(String filePath, Patient patient) {
        BufferedReader reader = null;
        try {
            // TRY: Normal processing flow
            reader = new BufferedReader(new FileReader(filePath));
            if (patient == null) {
                throw new IllegalArgumentException("Patient cannot be null");
            }
            String firstLine = reader.readLine();
            if (firstLine == null) {
                throw new IOException("Empty MRI file");
            }
            int width = Integer.parseInt(firstLine.split(",")[0]);
            int height = Integer.parseInt(firstLine.split(",")[1]);
            System.out.println("MRI dimensions: " + width + "x" + height);

        } catch (FileNotFoundException e) {
            // CATCH: Specific — file missing
            System.err.println("ERROR: MRI file not found: " + filePath);
            logToAuditSystem("Missing MRI file for patient: " + patient.getId());
            notifyRadiologist("Required MRI missing", patient.getId());

        } catch (IOException e) {
            // CATCH: General I/O error
            System.err.println("ERROR: Failed to read MRI file: " + e.getMessage());
            logToAuditSystem("I/O error reading MRI: " + e.getMessage());

        } catch (NumberFormatException e) {
            // CATCH: Corrupted data
            System.err.println("ERROR: Corrupted MRI data format");
            logToAuditSystem("Corrupted MRI data");

        } finally {
            // FINALLY: ALWAYS executes — close resources
            try {
                if (reader != null) {
                    reader.close();
                    System.out.println("MRI file closed successfully");
                }
            } catch (IOException e) {
                System.err.println("WARNING: Failed to close MRI file");
            }
        }
    }
}
```

✔ The **finally block** guarantees resource cleanup regardless of whether an exception occurred — critical for preventing file handle leaks in high-throughput medical imaging systems processing thousands of scans daily.

try-with-Resources — Java 7+ Automatic Resource Management

```
// RECOMMENDED: try-with-resources
public static void processMedicalReport(String filePath) {
    // Resources declared in try() are auto-closed
    try (BufferedReader reader =
        new BufferedReader(new FileReader(filePath));
        FileWriter writer =
            new FileWriter("processed_report.txt")) {

        String line;
        while ((line = reader.readLine()) != null) {
            writer.write(line.toUpperCase());
            writer.write("\n");
        }
        System.out.println("Report processed successfully");

    } catch (FileNotFoundException e) {
        System.err.println("Report file not found: " + filePath);
    } catch (IOException e) {
        System.err.println("I/O error: " + e.getMessage());
    }
    // No finally block needed — resources automatically closed
}
```

Why try-with-Resources Matters in Healthcare

→ Zero Resource Leaks

AutoCloseable resources are guaranteed to close even if an exception is thrown during processing — no manual finally block required.

→ Suppressed Exceptions

If both the try body and close() throw exceptions, the close() exception is suppressed and accessible via getSuppressed() — preserving the primary exception.

→ Cleaner Code

Eliminates nested finally blocks, reducing cognitive complexity in critical healthcare processing pipelines.

 In healthcare systems processing **50,000+ medical records daily**, resource leaks from improperly closed database connections can cascade into system-wide outages within hours.

throw vs. throws — A Critical Distinction

Keyword	Purpose	Location	Example
throw	Explicitly throw an exception instance at a specific point in code	Inside method body	throw new PatientNotFoundException (id);
throws	Declare that a method may propagate one or more checked exceptions to its caller	Method signature	public Patient find(String id) throws SQLException

```
class HospitalAdmissionSystem {
    // THROWS: Declares checked exceptions that may propagate
    public void admitPatient(Patient patient, String wardType)
    throws InsufficientStaffException, BedUnavailableException {

        // THROW: Guard clauses with explicit exception throwing
        if (patient == null) {
            throw new IllegalArgumentException("Patient cannot be null"); // Unchecked
        }
        if (patient.isAlreadyAdmitted()) {
            throw new PatientAlreadyAdmittedException(
                "Patient " + patient.getId() + " already admitted"); // Unchecked
        }
        if (!isBedAvailable(wardType)) {
            throw new BedUnavailableException(
                "No beds available in " + wardType); // Checked
        }
        if (!hasEnoughStaff(wardType)) {
            throw new InsufficientStaffException(
                "Insufficient nurses for " + wardType + " ward"); // Checked
        }
        patient.setAdmitted(true);
        System.out.println("Patient " + patient.getName() + " admitted to " + wardType);
    }
}
```

Custom Exceptions — Healthcare Domain Design

Well-designed custom exceptions carry domain-specific context, enabling precise error handling and HIPAA-compliant audit logging. The following hierarchy demonstrates four levels of sophistication:



Level 1 — Simple Custom Exception

PatientValidationException extends Exception — carries only a message. Suitable for straightforward validation failures such as missing required fields on patient intake forms.



Level 2 — Exception with Domain Data

MedicalRecordNotFoundException — carries patientId and recordType fields. Enables callers to display precise, actionable error messages without exposing PHI.



Level 3 — Unchecked with Safety Data

InvalidDosageException extends RuntimeException — carries prescribedDosage and maxSafeDosage. Immediately surfaces medication safety violations as programming errors.



Level 4 — Exception with Error Codes (HIPAA)

HipaaViolationException — carries structured error codes (H-1001 UNAUTHORIZED_ACCESS, H-1002 MISSING_CONSENT, H-1003 INCORRECT_MASKING) for regulatory audit trails.

Custom Exception Code — HIPAA-Compliant Design

```
// Custom exception with HIPAA error codes
class HipaaViolationException extends RuntimeException {
    private String errorCode;
    private String violatingField;

    public enum HipaaErrorCode {
        UNAUTHORIZED_ACCESS("H-1001"),
        MISSING_CONSENT("H-1002"),
        INCORRECT_MASKING("H-1003");

        private String code;
        HipaaErrorCode(String code) { this.code = code; }
        public String getCode() { return code; }
    }

    public HipaaViolationException(HipaaErrorCode errorCode, String violatingField) {
        super(String.format("HIPAA Violation [%s]: %s",
            errorCode.getCode(), violatingField));
        this.errorCode = errorCode.getCode();
        this.violatingField = violatingField;
    }
}

// Usage in PrescriptionService
class PrescriptionService {
    private static final double MAX_SAFE_PARACETAMOL = 4000.0; // mg/day

    public void prescribeParacetamol(Patient patient, double dosageMg) {
        if (dosageMg > MAX_SAFE_PARACETAMOL) {
            throw new InvalidDosageException(dosageMg, MAX_SAFE_PARACETAMOL);
        }
        if (patient.hasLiverDisease() && dosageMg > 2000) {
            throw new InvalidDosageException(dosageMg, 2000);
        }
    }

    public void accessMedicalRecord(String userId, Patient patient, String recordType)
        throws MedicalRecordNotFoundException {
        if (!userHasAccess(userId, patient)) {
            throw new HipaaViolationException(
                HipaaViolationException.HipaaErrorCode.UNAUTHORIZED_ACCESS,
                "Patient record: " + patient.getId());
        }
        if (!recordExists(patient.getId(), recordType)) {
            throw new MedicalRecordNotFoundException(patient.getId(), recordType);
        }
    }
}
```

 HIPAA Violation: Never include SSNs, dates of birth, or full patient names in exception messages. Use masked identifiers (patient IDs) only. Violations can result in fines of **\$100 to \$50,000 per record** exposed.

Exception Chaining — Preserving Root Cause Across Layers

Definition

Exception chaining (nested exceptions) allows wrapping one exception inside another, preserving the original cause for debugging across architectural layers.

Healthcare Analogy

A patient presents with a fever (symptom) caused by pneumonia (root cause). The fever "wraps" the pneumonia — treating the fever alone does not cure the pneumonia. Similarly, a

ClinicalDecisionException wraps a **DatabaseConnectionException** — fixing the UI error message alone does not resolve the database timeout.

Key Chaining Methods

Throwable(Throwable cause)

Constructor accepting a cause exception

Throwable(String msg, Throwable cause)

Constructor with message and cause

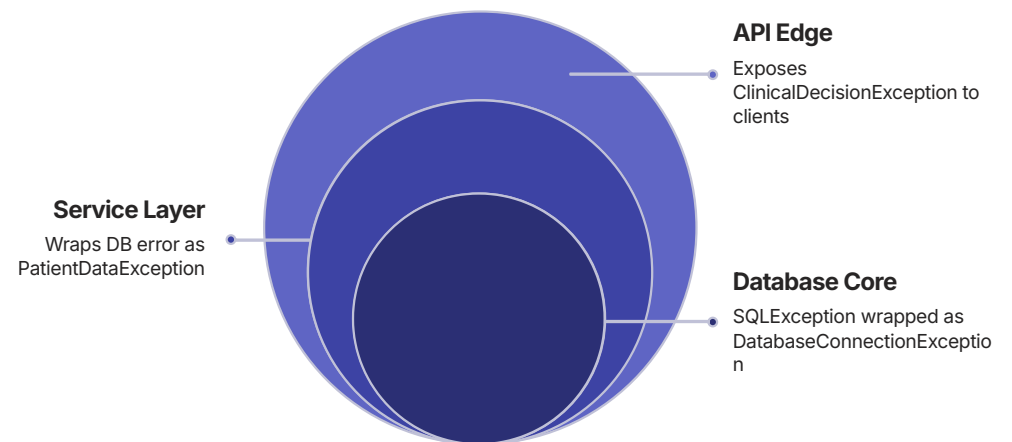
getCause()

Retrieve the wrapped cause exception

initCause(Throwable cause)

Set cause after construction

Three-Layer EHR Exception Chain



Each architectural layer adds contextual information while preserving the original root cause. The full chain is accessible via `getCause()` traversal, enabling precise root cause analysis in production incident investigations.

Exception Chaining — Complete EHR Code Example

```
class ElectronicHealthRecordSystem {

    // Layer 1: Database layer
    private Patient fetchFromDatabase(String patientId)
    throws DatabaseConnectionException {
    try {
    throw new SQLException("Connection timeout to EHR database");
    } catch (SQLException e) {
    throw new DatabaseConnectionException(
    "Failed to fetch patient " + patientId + " from EHR", e);
    }
    }

    // Layer 2: Service layer
    public Patient getPatientData(String patientId)
    throws PatientDataException {
    try {
    return fetchFromDatabase(patientId);
    } catch (DatabaseConnectionException e) {
    throw new PatientDataException(
    "Unable to retrieve clinical data for patient " + patientId, e);
    }
    }

    // Layer 3: Clinical decision layer
    public void makeDiagnosis(String patientId)
    throws ClinicalDecisionException {
    try {
    Patient patient = getPatientData(patientId);
    } catch (PatientDataException e) {
    throw new ClinicalDecisionException(
    "Cannot make diagnosis - patient data unavailable", e);
    }
    }
    }

    // Walking the exception chain
    try {
    ehr.makeDiagnosis("P999");
    } catch (ClinicalDecisionException e) {
    Throwable cause = e.getCause();
    int level = 1;
    while (cause != null) {
    System.out.println("Caused by (level " + level + "): "
    + cause.getClass().getSimpleName()
    + " - " + cause.getMessage());
    cause = cause.getCause();
    level++;
    }
    }

    /* OUTPUT:
    Caused by (level 1): PatientDataException - Unable to retrieve clinical data for patient P999
    Caused by (level 2): DatabaseConnectionException - Failed to fetch patient P999 from EHR
    Caused by (level 3): SQLException - Connection timeout to EHR database
    */
}
```

Re-throwing Exceptions — Five Patterns

1

Re-throw with Context

Catch `IOException` and re-throw with additional context: `throw new IOException("Failed while processing patient data", e);` — preserves original cause.

2

Exception Translation

Translate low-level `SQLException` to business exception: `throw new ServiceException("Database error during patient lookup", e);` — hides infrastructure details from callers.

3

Unchecked to Checked

Wrap `ArithmeticException` in a checked `Exception` when a calculation error must be explicitly handled by the caller in a dosage computation pipeline.

4

Conditional Re-throw

Re-throw `PatientNotFoundException` for regular users but silently handle for admin users — context-sensitive exception propagation based on caller identity.

5

Re-throw with finally

Capture the original exception reference before re-throwing, enabling the `finally` block to log the exception message during guaranteed cleanup execution.

- ❑ Exception translation (Pattern 2) is the most important pattern in layered healthcare architectures. It prevents low-level database implementation details from leaking into clinical API contracts, maintaining clean separation of concerns.

Real-World Scenario: Emergency Room Triage System

The following complete system integrates all exception handling concepts into a production-grade Emergency Room Triage application — demonstrating how checked exceptions, unchecked exceptions, custom exceptions, and exception chaining work together in a life-critical healthcare context.

Custom Exception Hierarchy

TriageException (base) → PatientNotFoundException, BedAssignmentException (checked).
CriticalPatientException (unchecked RuntimeException) for Code Blue scenarios.

Triage Level Enum

Five levels: RESUSCITATION (1), EMERGENCY (2), URGENT (3), SEMI_URGENT (4), NON_URGENT (5) — each mapped to appropriate ward types and bed assignments.

Multi-Exception Method

assessTriage() declares throws PatientNotFoundException, TriageException — demonstrating multi-exception declaration and exception chaining for vitals measurement failures.

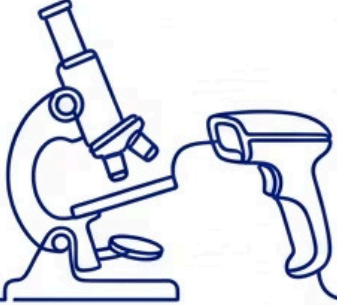
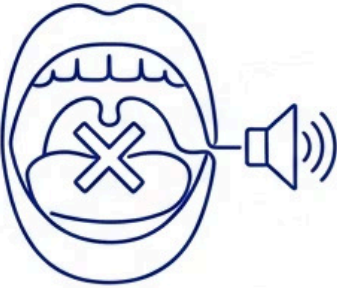
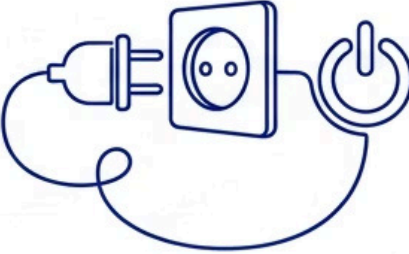
HIPAA Audit Logging

The finally block in processEmergencyPatient() guarantees every admission attempt is logged with timestamp, patient ID, triage level, and bed assignment — regardless of outcome.

Emergency Room Triage — Core Processing Code

```
class EmergencyTriageSystem {  
  
    public void processEmergencyPatient(String patientId, String name, int age) {  
        Patient patient = null;  
        TriageLevel triage = null;  
        Bed bed = null;  
  
        try {  
            // Step 1: Register  
            patient = registerPatient(patientId, name, age);  
  
            // Step 2: Assess (throws checked exceptions)  
            triage = assessTriage(patient);  
  
            // Step 3: Assign bed (throws checked exception)  
            bed = assignBed(patient, triage);  
  
            // Step 4: Code Blue if critical (throws unchecked)  
            if (triage == TriageLevel.RESUSCITATION) {  
                triggerCodeBlue(patient);  
            }  
            System.out.println(""
```

Best Practices — Healthcare Exception Handling

			
1. BE SPECIFIC Use specific exceptions, not generic Exception.	2. DON'T SILENCE Never swallow exceptions with empty catch blocks.	3. LOG CONTEXT Log full context including patient ID and timestamp.	4. USE RESOURCES Use try-with-resources for all I/O.
			
5. MASK PHI Mask sensitive PHI in exception messages.	6. BUILD HIERARCHY Build domain exception hierarchy.	7. DOCUMENT IT Document exceptions with @throws Javadoc.	8. FAIL FAST Fail fast with guard clauses.

These eight practices collectively ensure that exception handling in healthcare software meets both technical quality standards and regulatory compliance requirements under HIPAA and GDPR.

DO's and DON'Ts — HIPAA-Safe Exception Handling

✓ DO: Mask Sensitive Data

```
private String maskSSN(String ssn) {  
    return "XXX-XX-" + ssn.substring(7);  
}  
// Exception message: "Patient not found with SSN:  
XXX-XX-6789"
```

Never include full SSNs, dates of birth, or diagnosis codes in exception messages that may appear in logs or user interfaces.

✗ DON'T: Expose PHI in Exceptions

```
// HIPAA VIOLATION — never do this:  
throw new Exception(  
    "Patient SSN: 123-45-6789 not found");  
// This message could be logged, displayed,  
// or transmitted — exposing protected health info
```

A single logged exception containing unmasked PHI constitutes a reportable HIPAA breach, triggering mandatory notification obligations.

✓ DO: Build Domain Hierarchy

```
class MedicalException extends Exception {}  
class PatientException extends MedicalException {}  
class TreatmentException extends MedicalException {}  
{}  
class BillingException extends MedicalException {}
```

A domain exception hierarchy enables callers to catch at the appropriate level of specificity — catch `MedicalException` for all medical errors, or `PatientException` for patient-specific issues only.

✓ DO: Document with @throws

```
/**  
 * Admits a patient to the hospital.  
 * @param patient the patient to admit (must not be  
 null)  
 * @throws BedUnavailableException if no beds  
 available  
 * @throws InsufficientStaffException if nurses  
 insufficient  
 * @throws IllegalArgumentException if patient is null  
 */  
public void admitPatient(Patient patient)  
    throws BedUnavailableException,  
    InsufficientStaffException
```

Javadoc `@throws` annotations form the contract between method author and caller — essential for team-based healthcare software development.

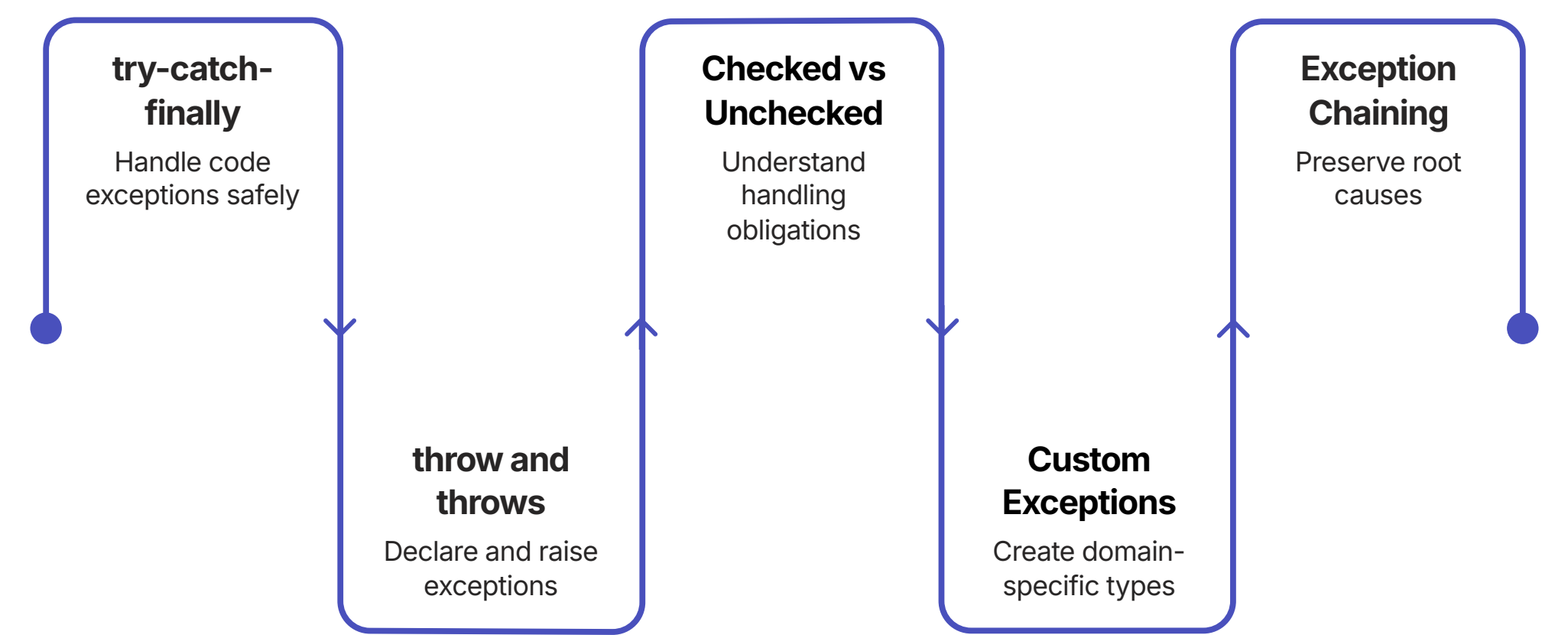
Quick Reference — Exception Hierarchy Summary

Type	Superclass	Checked?	When to Use	Healthcare Example
Error	Error	No	JVM failures — do not catch	Power outage, JVM crash, MRI firmware failure
Checked Exception	Exception	Yes	Expected, recoverable conditions	Patient not found, database connection down
Unchecked Exception	RuntimeException	No	Programming bugs — fix the code	Null patient reference, invalid dosage value

Common Exceptions Quick Guide

Exception	Type	Common Cause	Healthcare Context
NullPointerException	Unchecked	null reference accessed	Patient object null in admission flow
IllegalArgumentException	Unchecked	Invalid argument passed	Negative dosage prescribed
IllegalStateException	Unchecked	Invalid object state	Surgery already in progress
IOException	Checked	I/O operation failed	Cannot read MRI file from PACS
SQLException	Checked	Database error	Cannot connect to EHR database
ClassNotFoundException	Checked	Class missing at runtime	Medical device driver not in classpath

Learning Path Summary & Key Takeaways



Throwable Hierarchy

Throwable → Exception/Error → Checked/Unchecked provides a structured taxonomy. Every Java exception has a precise place in this hierarchy with defined handling obligations.

Checked = Predictable

PatientNotFoundException represents a foreseeable business condition. The compiler enforces handling — your first automated patient safety check.

Unchecked = Bugs

InvalidDosageException represents a programming defect. Fix the validation logic — do not mask the bug with a catch block.

Chaining = Root Cause

Exception chaining across architectural layers preserves the original SQLException beneath a ClinicalDecisionException — enabling precise incident investigation.

In healthcare software, you cannot have a NullPointerException crashing the EMR system during a Code Blue emergency. Proper exception handling is not optional — it is a patient safety imperative. — **Talenciaglobal**